

GAME PROGRAMMING COURSE — VERSION CONTROL

Git でバージョン管理

成り立ち・内部の仕組み・ふだん使わない機能まで

対象：高等部3年（全3コマ）

GIT/
GITHU
B

LESSON 1

成り立ち： 集中型から分散型へ

目標：Git がなぜ「分散型」として作られたのかを、歴史から説明できる

バージョン管理がないと、こうなる

Assets / Scripts

Player.cs

Player_修正.cs

Player_最終.cs

Player_最終2.cs

Player_最終_本当に最終.cs

Player_0610_田中.cs

Player_バックアップ(1).cs

どれが最新かわからない

ファイル名で管理すると、名前のルールがすぐに崩れる。

何を変えたか思い出せない

昨日は動いていたのに今日は動かない。どこを変えたせいか追えない。

人の作業を上書きして消す

USB や共有フォルダで受け渡すと、後から保存した人の内容だけが残る。

バージョン管理システム（VCS）とは

ファイルの変更を、「いつ・誰が・何を・なぜ」とセットで記録していく仕組み。

① 履歴を残す

変更のたびに記録を積み重ねる。ファイル名に日付や「最終」を付ける必要がなくなる。

② 過去に戻す・比べる

「動いていた時点」に戻せる。2つの時点の差分を見て、バグの原因を絞り込める。

③ 複数人の作業を合わせる

別々の人の変更を1つにまとめる。ぶつかった場所は、仕組みが教えてくれる。

用語	意味
リポジトリ	ファイルと、その変更履歴をまとめて保管する場所
コミット	変更を履歴として記録すること。記録された1件1件もコミットと呼ぶ
作業コピー	リポジトリから取り出した、実際に編集するファイル一式

バージョン管理の3つの世代

第1世代 ローカル型

RCS (1982年)

1台の PC の中だけで履歴を管理する。

他の人と共有する仕組みがない。



第2世代 集中型

CVS (1990年)

Perforce (1995年)

Subversion (2000年)

1台のサーバーに履歴を置き、全員がそこへつながる。



第3世代 分散型

Git (2005年)

Mercurial (2005年)

全員が履歴を丸ごと自分の PC に持つ。

今日の主役はここ。

POINT

ちがいは「履歴（リポジトリ）をどこに置くか」。次のページから、集中型と分散型を図で比べます。

集中型の中での進化：CVS から Subversion へ

CVS（1990年～）の弱点

- 履歴はファイル**1つ**ずつばらばらに持つ
- **10個**のファイルをコミット中に通信が切れると、**5個**だけ入った状態で残る
- ファイル名の変更や移動を追えない
- ブランチとマージが難しく、避けられていた

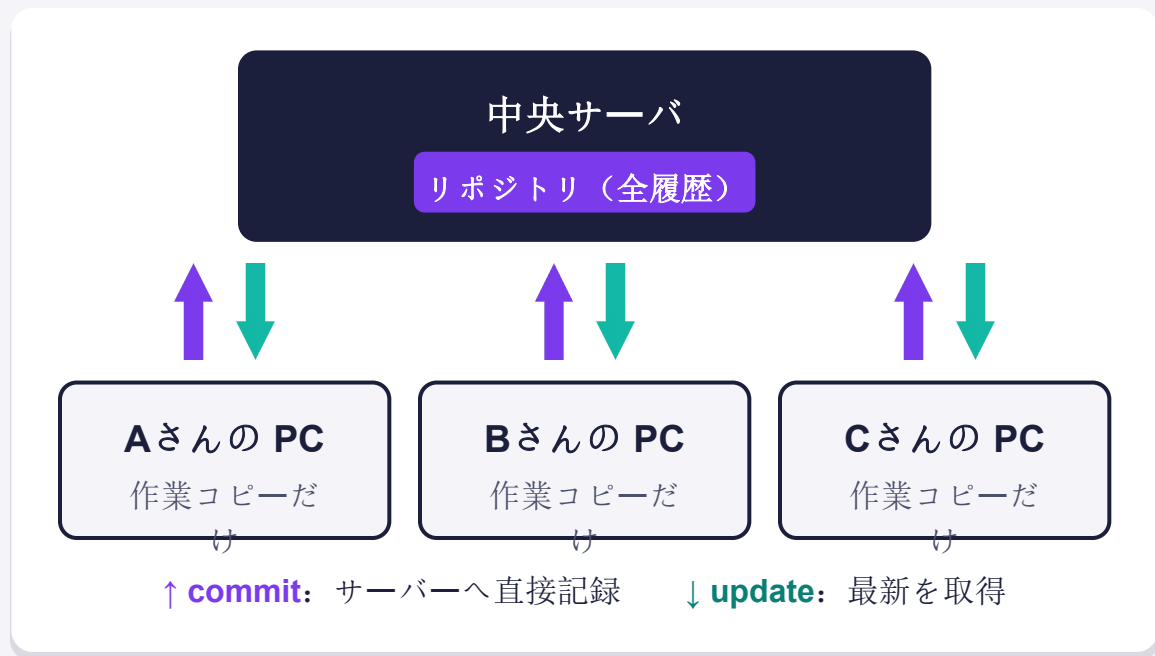
Subversion（2000年～）の改良

- 「**CVS** の作り直し」を目標に開発された
- コミットは**全部入るか、全部入らないか**（アトミック）
- リポジトリ全体に連番が付く（リビジョン **1、2、3...**）
- フォルダや名前の変更も履歴に残る

POINT

どちらも「履歴はサーバーに**1つ**」という点は同じ。連番が使えるのは、番号を配るサーバーが**1つ**だからです。この点は LESSON 2 でもう一度出てきます。

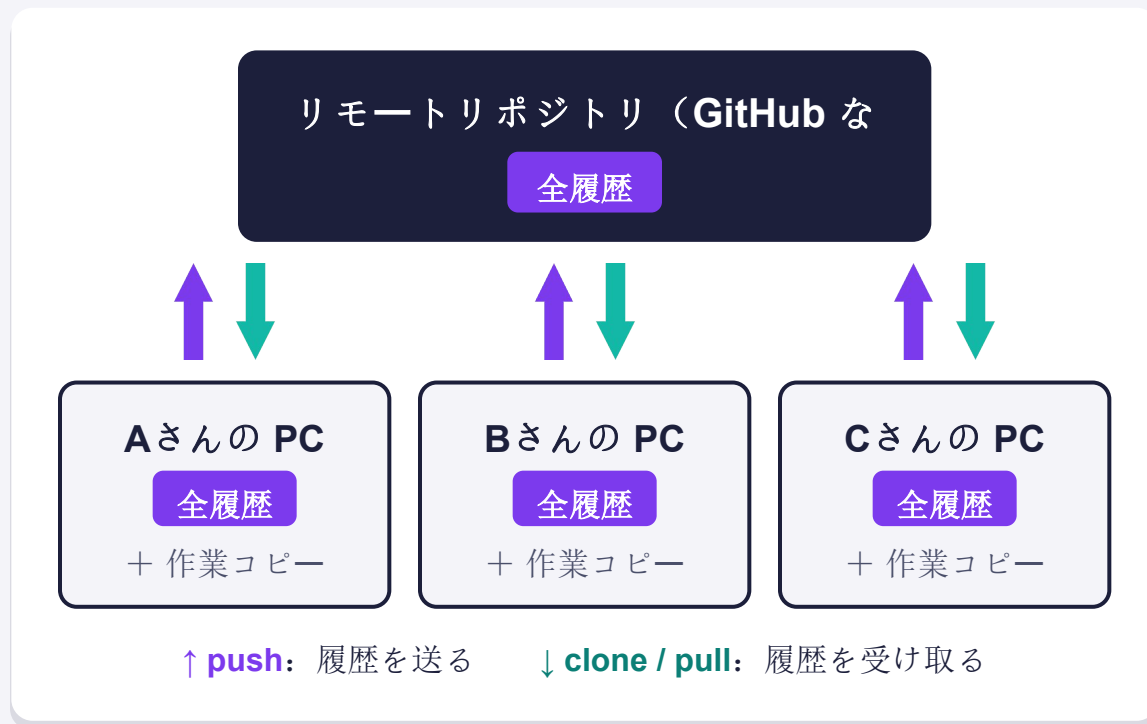
集中型：履歴はサーバーに1つだけ



- 履歴を持っているのはサーバーだけ
- 手元にあるのは「今の状態」のファイル一式
- **commit** = サーバーへの記録。ネットワークが必須
- 過去の履歴を見るのも、サーバーに問い合わせる
- サーバーが止まると、全員の作業が止まる

代表例: Subversion (SVN)、Perforce

分散型：全員が履歴を丸ごと持つ



- 全員の **PC** にリポジトリが丸ごと入っている
- **commit** = 自分の **PC** の中に記録。オフラインでもできる
- 履歴を見る・過去に戻すのも手元で完結し、速い
- 他の人に渡したいときだけ **push** する
- サーバーが壊れても、誰かの **PC** から復元できる

代表例: Git、Mercurial

同時編集への2つの答え：ロックとマージ

2人が同じファイルを同時に変えたい。このとき、仕組みとしてどう扱うか。

ロック方式（先に防ぐ）

ロック → 編集 → 解除

- 編集中は、他の人がそのファイルを変更できない
- ぶつかること自体が起きない
- 画像・3Dモデルなど、合体できないファイルに強い
- 弱点：待ち時間が出る。解除を忘れると全員が止まる

マージ方式（後で合わせる）

コピー → 編集 → 合体

- 全員が同時に、自由に編集してよい
- 待ち時間がない
- コードなどのテキストは、別の行なら自動で合体
- 弱点：同じ場所を変えると、人が手で解決する（コンフリクト）

POINT

サーバーが1つの集中型はロックの管理が得意。Git は マージ方式が基本で、LESSON 2 でその仕組みを扱います。

集中型と分散型の比較

項目	集中型（SVN / Perforce）	分散型（Git）
履歴の場所	サーバーだけ	全員の PC + サーバー
コミット先	サーバー（ネットワーク必須）	自分の PC（オフラインでも可）
履歴を見る・比べる	サーバーに問い合わせる	手元で完結するので速い
サーバーが壊れたら	バックアップがなければ履歴を失う	誰かの PC から復元できる
ブランチ	重く、作る回数は少なめ	軽く、気軽に作る前提
同時編集	ロックで防ぐ運用が得意	マージで合わせるのが基本
大きなバイナリ	得意（必要な分だけ取得できる）	苦手（Git LFS で補う）
権限の設定	フォルダ単位で細かく決められる	基本はリポジトリ単位
覚えること	少なめ	多め（場所とブランチの概念）

集中型が今も使われる場面

大規模開発と Perforce

- テクスチャ・モデル・ムービーで、プロジェクトが数百 GB になる
- 全履歴を全員の PC に置くのは現実的でない
- 自分の担当フォルダだけ取得できる
- アーティストがロックを使って安全に作業できる

Git 側の対策

- **Git LFS:** 大きなファイルの本体を別の場所に置き、リポジトリには「引換券」だけを入れる
- **Git LFS** のロック機能で、編集集中のファイルを知らせる
- 必要な部分だけ取得する機能もある

POINT

優劣ではなく向き・不向き。学校のチーム制作の規模なら Git と GitHub で十分に回せます。就職先で Perforce や SVN に出会っても、考え方は今日の内容がそのまま使えます。

Git と GitHub は別のもの

Git (ツール)

- 自分の PC で動く、バージョン管理のソフト
- 無料のオープンソース
- ネットワークがなくても、1人でも使える
- 履歴の記録・ブランチ・マージを担当する

GitHub (サービス)

- Git のリモートリポジトリを置ける Web サービス
- Pull Request や Issue など、チーム作業の機能がある
- 同じ役割のサービス: GitLab、Bitbucket
- Git がなければ成り立たない（逆は成り立つ）

「分散型」でも、チームでは集合場所になるリモートを1つ決めるのがふつう。

分散型の意味は「サーバーがない」ではなく、「サーバーがなくても全員が完全な履歴を持っている」こと。

LESSON 2

Git の 内部の仕組み

目標: コミット・ブランチ・マージが、内部で何をしているかを説明できる

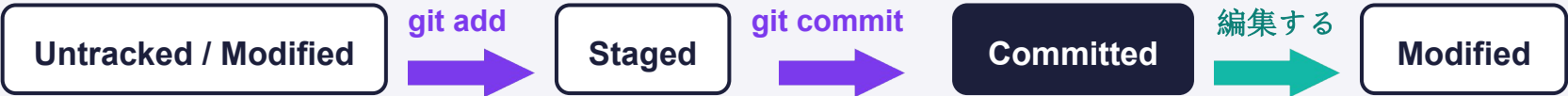
Git の4つの場所



- **commit** は「ローカルリポジトリへの記録」。この時点では、他の人にはまだ見えない
- push して初めて、リモートに届く
- .git という隠しフォルダが、ステージングエリアとローカルリポジトリの正体

ファイルの4つの状態

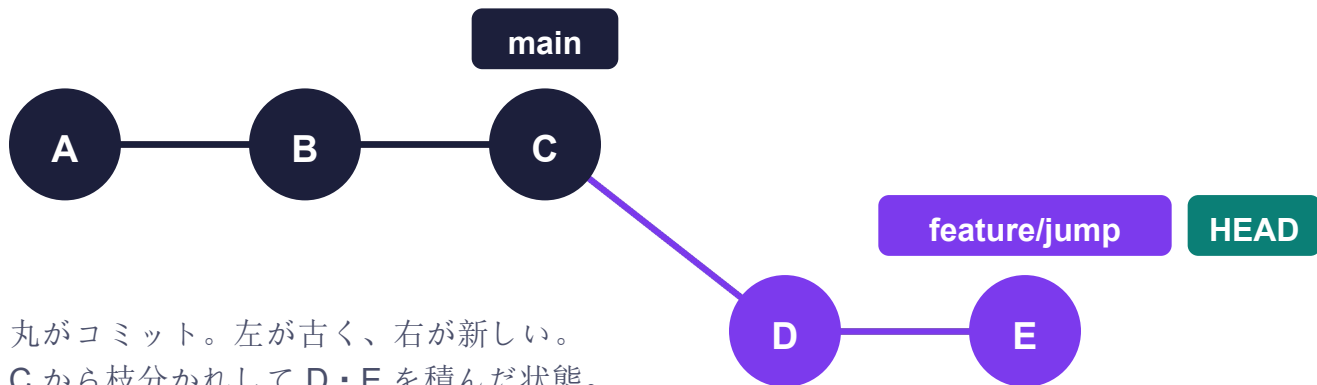
状態	意味	git status での表示
Untracked（未追跡）	Git がまだ管理していない、新しいファイル	Untracked files
Modified（変更あり）	前回のコミットから変更したが、まだ add していない	Changes not staged for commit
Staged（ステージ済み）	add して、次のコミットに入る予定になっている	Changes to be committed
Committed（記録済み）	履歴に記録されていて、変更がない	nothing to commit, working tree clean



POINT

今どの状態かわからなくなったら、まず **git status**。次に打つコマンドのヒントも表示されます。

ブランチとは



- 履歴を枝分かれさせる仕組み
- 実体は「あるコミットを指すラベル」。作るのは一瞬
- **main:** いつでも動く、全員の基準
- 作業用ブランチ: 機能ごとに作る
- **HEAD:** 自分が今いる場所

POINT

作りかけの機能を **main** に混ぜずに進められる。集中型ではブランチの作成や切り替えにサーバーとのやりとりが必要だが、**Git** は手元だけで完結するので気軽に作れる。

origin/main : 手元にあるリモートの写し



- **origin/main** は「最後に通信したときのリモートの **main**」。今のリモートの状態ではない
- `git status` の「ahead 2」「behind 3」は、**main** と **origin/main** を比べた結果。通信はしていない
- `pull` は「`fetch` で写しを更新」と「写しを自分のブランチに `merge`」の2段階
- リモートで消されたブランチの写しは残り続ける → `git fetch --prune` で掃除する

POINT

「`status` では最新と出たのに `push` で断られた」の原因はこれ。**fetch** するまで、手元の **Git** は他の人の **push** を知らない。分散型では、通信は自分が頼んだときだけ起きます。

リモートとのやりとり：fetch・pull・push

git fetch

受け取るだけ

リモートの新しい履歴を、ローカルリポジトリに取ってくる。作業中のファイルは変わらない。

git pull

受け取って、合わせる

fetch + merge。取ってきた履歴を自分の作業に合体させ、ファイルも最新になる。

git push

送る

自分のコミットをリモートへ送る。送れるのはコミット済みのものだけ。

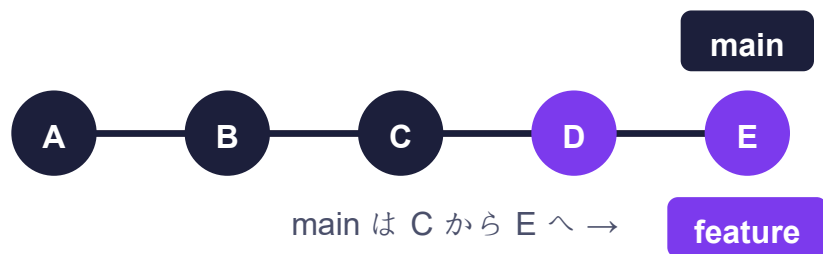
ルール：作業を始める前に **pull**、区切りがついたら **push**

注意

push が **rejected** と表示されたら、リモートに自分の知らないコミットがある合図。先に **git pull** してから、もう一度 push する。

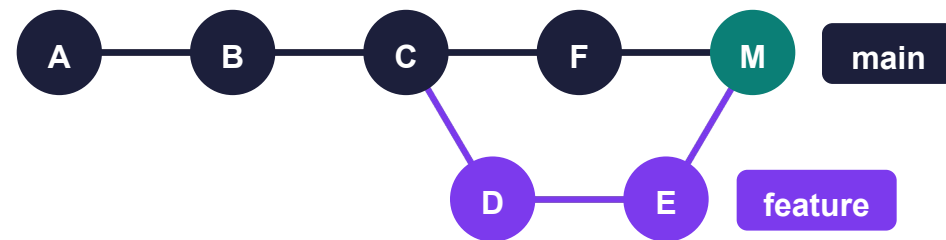
マージ：枝分かれした履歴を1つにする

早送り (fast-forward)



- 枝分かれの後、**main** が進んでいないとき
- **main** のラベルを先へ進めるだけ
- 新しいコミットは作られない

マージコミット



- 枝分かれの後、**main** も進んでいるとき (F)
- 2つを合わせた新しいコミット M ができる
- M は親を2つ持つ (F と E)

どちらになるかは **Git** が自動で決める。チーム開発では **main** が進んでいることが多いので、右の形をよく見る。

3ウェイマージ：Git はどう判断しているか

2つのブランチを見比べるだけでは、「どちらが変えたのか」がわからない。Git は枝分かれした地点（共通の祖先）を加えた3つを比べる

共通の祖先	自分（main）	相手（feature）	Git の判断
jumpPower = 5	8	5	自分だけが変えた → 8 を採用
jumpPower = 5	5	12	相手だけが変えた → 12 を採用
jumpPower = 5	8	8	同じ変更をした → 8 を採用
jumpPower = 5	8	12	両方が別々に変えた → コンフリクト

- この比較を、ファイルの**変更のかたまりごと**に行う
- だから、同じファイルでも離れた場所の変更は自動で合体できる
- 共通の祖先は、コミットの「親」をたどって **Git** が自動で見つける

POINT

Git が合体させているのは**文字の並び**で、プログラムの意味は見ていない。コンフリクトが出なくても、マージ後に動くかは人が確かめる必要があります。

コンフリクト：同じ場所を2人が変えた

別の行の変更は、Git が自動で合体する。同じ行を別々に変えたときだけ、Git は判断を人に任せる。

C# Player.cs (マージ直後)

```
1 void Jump()
2 {
3     <<<<<< HEAD
4     float jumpPower = 8.0f;
5     =====
6     float jumpPower = 12.0f;
7     >>>>>> feature/jump
8     rb.AddForce(Vector2.up * jumpPower);
9 }
```

<<<<<< HEAD から =====

今いるブランチ（取り込む側）の内容

===== から >>>>>>

取り込もうとしたブランチの内容

Git が、ぶつかった場所に目印を書き込んでくれている。エラーで壊れたわけではない。

意識しないと気づかない

Git が記録しないもの

空のフォルダ

記録するのはファイルだけ。Unity は空のフォルダにも **.meta** を作るので、相手の PC では「**.meta** だけあってフォルダがない」状態になり、警告が出る。

名前を変えた、という事実

「削除」と「追加」として記録され、中身が似ていればあとから推測して「名前変更」と表示する。名前変更と大きな書き換えを同時にすると、履歴がつながらなくなる。

大文字と小文字だけの変更

Windows は `player.cs` と `Player.cs` を同じ名前として扱う。エクスプローラーで直しても Git に伝わらないことがある。**git mv** で変える。

add していない変更

一度も `add`・`commit` していない編集は、**Git** のどこにも入っていない。上書きや削除をすると、**Git** では取り戻せない。

POINT

上の2つは、内部構造から説明できる。フォルダ（**tree**）は中身の一覧表なので空では作れず、ファイル名は **blob** ではなく **tree** 側にあるので、名前変更という記録の置き場がない。

意識しないと気づかない

.gitignore の細かいルール

.gitignore

```
1  # 先頭の / は「一番上の階層の」という意味
2  /Library/
3  /Temp/
4
5  # / なしは、どの階層にあっても無視
6  .vs/
7
8  # * は「何でも」。拡張子でまとめて無視
9  *.csproj
10
11 # ! は例外。これだけは無視しない
12 !Tools.csproj
```

- 上から順に読まれ、後に書いた行が優先される
- 末尾の / はフォルダだけに一致する
- .gitignore 自体もコミットして、全員で共有する
- 自分だけの無視設定は .git/info/exclude に書く（共有されない）

注意

すでにコミットしたファイルには効かない。無視できるのは、まだ追跡していないファイルだけ。後から外すには **git rm --cached** ファイル名 で追跡をやめてから、コミットする。

POINT

「なぜか無視される / されない」ときは **git check-ignore -v** ファイル名。どのファイルの何行目のルールが効いているかを表示します。

意識しないと気づかない

.gitattributes と改行コード

.gitattributes

```
1  # テキストの改行コードを Git にそろえさせる
2  * text=auto
3
4  # バイナリとして扱う（差分もマージもしない）
5  *.png binary
6
7  # Git LFS で管理する
8  *.psd filter=lfs diff=lfs merge=lfs -text
9
10 # シーンは Unity のツールでマージする
11 *.unity merge=unityyamlmerge
```

- ファイルの種類ごとの扱いを決める設定ファイル
- リポジトリに入るので、全員に同じ設定が効く
- .gitignore は「入れるか」、.gitattributes は「どう扱うか」

改行コードの問題

改行を表す文字は、Windows が CRLF、Mac と Linux が LF。そろえていないと、**1行も直していないのに全行が「変更あり」**になる。

POINT

差分が「全行変更」になっていたら、まず改行コードを疑う。本当の変更が埋もれて、レビューもマージもできなくなります。

Unity プロジェクトで管理するもの

コミットする

- Assets/
- Packages/
- ProjectSettings/
- .gitignore
- ***.meta** (すべて)

コミットしない

- Library/
- Temp/ Obj/ Logs/
- UserSettings/
- Build/
- *.csproj *.sln

注意

.meta ファイルは必ずコミットする。アセットの ID が入っていて、なくなるとシーンやプレハブの参照が外れる (Missing になる)。

POINT

Edit > Project Settings > Editor で、Asset Serialization の Mode が **Force Text** になっているか確認する。